

Sicheng Ouyang

sicheng.ouyang@uwaterloo.ca | (647) 531-8488 | GitHub: [carols12352](https://github.com/carols12352) | LinkedIn: [sicheng-ouyang](https://www.linkedin.com/in/sicheng-ouyang) | sicheng.dev

EDUCATION

University of Waterloo, Waterloo, ON

Sep 2025 – Present

Bachelor of Software Engineering, Expected Apr 2030

Competitive Programming: USACO Gold Division (2024) · CCC Junior Distinction (2024)

EXPERIENCE

WATcloud (WATonomous)

Sep 2026 – Present

Infrastructure & DevOps Engineer

Waterloo, ON

- Refactored Terraform CI across **5 services** into separate render and apply stages, hashing rendered configuration and provisioner code to skip unchanged applies since the last successful run while periodically reconciling external drift.
- Achieved a **53–94% Terraform apply skip rate** per service and reduced average CI runtime by **50%** by detecting configuration and provisioner hash changes, with periodic reconciliation to correct drift from external or manual changes.
- Maintained WATcloud's **Next.js** website with machine card UI improvements and CI link checker updates, caching crawl results to check broken links and page anchors without repeat HTTP requests during validation.

Mui Scientific

May 2026 – Aug 2026

Software Engineer

Mississauga, ON

- Cut audit preparation from **8 hours to 30 minutes** by automating ERP-to-inventory synchronization with vendor REST APIs, custom parsers, and SQL workflows, preserving traceability across **10,000+ serial/batch records**.
- Reduced routine manual checks by approximately **80%** through validation rules and mismatch alerts spanning procurement, warehouse, and sales records.
- Modernized **30+** corporate pages with typed Next.js templates and a crypt/HMAC-secured content API, enabling staff to schedule notices while achieving a **95+** Lighthouse performance score.

Tencent Music Entertainment Group

Jul 2024 – Aug 2024

Machine Learning Engineer

Shenzhen, China

- Evaluated 10 speech-generation models across 20+ controlled experiments using standardized Mel-spectrogram and F0 preprocessing, eliminating underperforming combinations and selecting the final singing-voice generation model.
- Optimized inference and vocal-simulation parameters through iterative tuning, raising an internal speaker consistency metric by 20% and MOS from **3.43** to **3.63**.

PROJECTS

The Cobalt Guide | [Next.js](#) · [PostGIS](#) · [MapLibre](#) · [Redis](#)

[Demo](#) [GitHub](#)

- Built a community-maintained map of **58,559 Canadian merchant locations** where users can find businesses, share Amex Cobalt reward multipliers, and report inaccurate listings using a Next.js interface with MapLibre clustering.
- Optimized map and search queries with PostGIS and GiST indexes, achieving p95 latencies of **65 ms and 45 ms**, respectively, across 30 production samples with warm CDN caches.
- Implemented community reporting and moderation with transactional PostgreSQL RPCs, Supabase RLS, and cache invalidation after commits, testing critical paths with **266 Vitest tests** alongside live database and Playwright suites.

AI Sourcing Workspace | [FastAPI](#) · [Next.js](#) · [OpenAI](#) · [SQLAlchemy](#)

- Built a **FastAPI and Next.js** application for extracting bills of materials from engineering documents and researching suppliers from company URLs, bringing part records, company profiles, and outreach drafts into one workspace.
- Combined rule-based parsing with vision-model extraction for **XLSX, PDF, and image** files to consolidate part quantities, materials, and tolerances across documents, preserving source links so users can review each extracted record.
- Automated company and contact research through sitemap crawling and LLM extraction with rule-based fallbacks, storing results and extraction history in **SQLAlchemy/SQLite** while blocking private-network URLs and redirects.

Relay Messaging | [FastAPI](#) · [Next.js](#) · [WebSocket](#) · [PostgreSQL](#)

[GitHub](#)

- Built a real-time chat application with **FastAPI, Next.js, and WebSockets**, combining persistent PostgreSQL message history with invitation-based access, JWT authentication, and email verification.
- Implemented automatic message recovery after reconnects and cursor-based history pagination so users can pick up conversations after a disconnect, with rate limits and per-user connection caps to control WebSocket usage.

SKILLS

Languages: Python, TypeScript, JavaScript, Java, C++, SQL

Frameworks & Libraries: FastAPI, React, Next.js, PyTorch

Data & Infrastructure: PostgreSQL, PostGIS, MySQL, Redis, Supabase, Docker, Linux, GitHub Actions, Slurm